

Elasticsearch OOM Crash — Incident Summary

What Happened

Elasticsearch kept crashing and stopping on its own, causing the entire SOC to go offline. The Elastic Agent on the SOC instance couldn't reach Elasticsearch on port 9200, and the EC2 agents in the Main VPC lost connection to Fleet Server on port 8220.

Root Cause

A typo in the `.env` file:

```
ES_LOCAL_JAVA_OPTS="-Xms512m -Xmx512g"
```

`-Xmx512g` told the JVM to allow up to **512 gigabytes** of heap memory. On a `t2.medium` with only 4GB RAM, the JVM thrashed the garbage collector trying to manage an impossible memory allocation — spending 7 seconds out of every 10 seconds just doing garbage collection — until the OS killed the process with **exit code 137** (OOM kill).

Why It Happens

Docker doesn't validate JVM heap flags before starting the container. Elasticsearch started fine, loaded all its data, then progressively ran out of usable memory as more logs were ingested. The garbage collector eventually hit a death spiral and the OS terminated the process to protect system stability.

Diagnostic Commands Used

Check which containers are running

```
docker ps
```

Check all containers including stopped ones

```
docker ps -a | grep es-local-dev
```

Read the crash logs to find the exit reason

```
docker logs es-local-dev --tail 50
```

Check system memory availability

```
free -h
```

Check what ports are actually listening

```
sudo netstat -tulpn | grep -E "5601|9200"
```

Read the environment config to find the heap setting

```
cat ~/elastic-start-local/.env
```

Read the docker compose file to understand the config structure

```
cat ~/elastic-start-local/docker-compose.yml
```

The Fix

Edit the .env file

```
nano ~/elastic-start-local/.env
```

Changed from (typo - 512 gigabytes):

```
ES_LOCAL_JAVA_OPTS="-Xms512m -Xmx512g"
```

Changed to (correct - 512 megabytes):

```
ES_LOCAL_JAVA_OPTS=-Xms512m -Xmx512m
```

Restart the stack

```
docker compose down
```

```
docker compose up -d
```

Lesson Learned

Always double-check JVM heap flags — **m** for megabytes, **g** for gigabytes. One wrong letter took down the entire SOC. In production you'd also set up a **CloudWatch alarm on the EC2 instance's memory metric** so this gets caught automatically before everything goes offline.

Heap Memory — What Is It?

Think of your computer's memory (RAM) like a desk. When you're working, you put things on the desk to use them. When you're done, you clear them off.

Heap memory is the portion of RAM that Java programs (like Elasticsearch) reserve exclusively for themselves — like booking a section of the desk just for one person.

Your t2.medium has 4GB RAM total

└— Ubuntu OS needs ~500MB

└— Docker needs ~200MB

└— Kibana needs ~500MB

└— Elastic Agent needs ~300MB

└— Elasticsearch heap = what's left (~512MB is safe)

-Xms512m = start with 512MB reserved (minimum) **-Xmx512m** = never use more than 512MB (maximum)

When we accidentally set **-Xmx512g** (512GB), Elasticsearch tried to manage a heap 128x larger than your entire server — like telling one person they can use a desk the size of a football field. The OS panicked and killed it.

Where Elasticsearch Stores Its Data

Elasticsearch data lives in a Docker volume, not a regular folder. Check its size:

See all Docker volumes

```
docker volume ls
```

Check the Elasticsearch volume size

```
docker system df -v | grep elasticsearch
```

The actual data is stored here on disk:

Find where Docker volumes live

```
sudo du -sh /var/lib/docker/volumes/
```

```
sudo du -sh /var/lib/docker/volumes/elastic-start-local_dev-elasticsearch/
```

How to Free Up Space

Option 1 — Delete old log indices in Kibana (safest, recommended):

Go to **Kibana** → **Stack Management** → **Index Management**

You'll see all your indices listed with their sizes. Old ones look like:

```
.ds-logs-aws.cloudtrail-default-2026.02.21-000001
```

```
.ds-logs-aws.vpcflow-default-2026.02.21-000001
```

Click any old index → **Manage** → **Delete index**. This removes old log data while keeping Elasticsearch running cleanly.

Option 2 — Check what's eating the most space first:

Check overall disk usage on the SOC instance

```
df -h
```

Check Docker's total footprint

```
docker system df
```

See breakdown by volume

```
docker system df -v
```

Option 3 — Delete ALL Elasticsearch data and start fresh (nuclear option):

Only do this if you want a completely clean slate:

```
docker compose down
```

```
docker volume rm elastic-start-local_dev-elasticsearch
```

```
docker volume rm elastic-start-local_dev-kibana
```

```
docker compose up -d
```

⚠ This wipes everything — all logs, dashboards, integrations. You'd have to reconfigure everything from scratch.

My Recommendation for Your Lab

For a learning environment, **Option 1** is best. Go to Index Management in Kibana and delete indices older than a week. Keeps the disk clean without losing your configuration.

What does `df -h` show for your current disk usage on the SOC instance?